GLOBAL ACADEMY OF TECHNOLOGY

Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi
Rajarajeshwari Nagar, Bengaluru – 560 098

GLOBAL ACADEMY
OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

A Mini Project Report on

CONTRACT CLAUSE TYPE IDENTIFIER

Project ID: MP-C38

Submitted in partial fulfilment of the requirements of the course

LARGE LANGUAGE MODELS (ISE23704C)

VII Semester, Bachelor of Engineering in Information Science and Engineering

Submitted by

SUHANI S RAO **1GA23IS166** 7B

SDG 16: Peace, Justice and Strong Institutions |
SDG 8: Decent Work and Economic Growth

Under the guidance of

[Name of the Course Faculty / Guide]
[Designation], Dept. of ISE, GAT

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
GLOBAL ACADEMY OF TECHNOLOGY, BENGALURU
Academic Year 2026–27

GLOBAL ACADEMY OF TECHNOLOGY

Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi
Rajarajeshwari Nagar, Bengaluru – 560 098

Department of Information Science and Engineering

GLOBAL ACADEMY
OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

C E R T I F I C A T E

This is to certify that the mini project entitled **"Contract Clause Type Identifier"** (Project ID: **MP-C38**) is a bonafide work carried out by **SUHANI S RAO**, USN **1GA23IS166**, of VII Semester, Section 7B, in partial fulfilment of the requirements of the course **Large Language Models (ISE23704C)** for the award of the degree of Bachelor of Engineering in Information Science and Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2026–27. The mini project report has been approved as it satisfies the academic requirements prescribed for the course.

[Name of Course Faculty / Guide]	**[Name of the HOD]**
[Designation]	Professor and Head
Dept. of ISE, GAT	Dept. of ISE, GAT

ABSTRACT

Legal contracts contain many clauses, and manually labelling each clause (for example governing law, notices or counterparts) is slow. This project built a Contract Clause Type Identifier that predicts the type of a clause from its text. The LEDGAR dataset from the LexGLUE benchmark was used. The 10 most frequent clause types were selected and a balanced subset of 800 clauses per class was prepared (70% training, 15% validation, 15% test, random seed 42). The GPT-2 124M model from the course was re-implemented, pretrained weights were loaded, and a new 10-class linear classification head was added. The head, the final layer norm and the last transformer blocks were fine-tuned with AdamW and cross-entropy loss, and the best checkpoint was chosen by validation macro-F1. Final test accuracy and macro-F1 are PENDING until train.py is run. A Gradio application classifies a single clause or every clause of an uploaded PDF/TXT contract and exports the results as CSV. The system is an academic prototype and not legal advice.

Keywords: GPT-2, fine-tuning, clause classification, LEDGAR, legal NLP, Gradio

CONTENTS

ABSTRACT.....i
CONTENTS.....ii
CHAPTER 1 INTRODUCTION.....1
1.1 Problem Statement.....1
1.2 Objectives.....1
1.3 Category and Approach.....1
CHAPTER 2 DATASET AND PREPROCESSING.....2
2.1 Dataset Description.....2
2.2 Preprocessing Steps.....2
2.3 Tokenization and Data Loader.....2
CHAPTER 3 MODEL AND TRAINING SETUP.....4
3.1 Model Architecture.....4
3.2 Changes Made for This Application.....4
3.3 Training Settings.....4
CHAPTER 4 RESULTS AND EVALUATION.....6
4.1 Before and After Training.....6
4.2 Project-Specific Tasks.....6
4.3 Error Analysis.....7
CHAPTER 5 APPLICATION.....8
5.1 Application Interface.....8
5.2 Sample Inputs and Outputs.....8
CHAPTER 6 CONCLUSION AND LIMITATIONS.....9
6.1 Conclusion.....9
6.2 Limitations and Ethical Considerations.....9
6.3 Future Enhancements.....9
REFERENCES.....10
APPENDIX A — CODE, NOTEBOOK LINK AND COMPLETE SOURCE CODE….11
A.1 Links (both required).....11
A.2 Steps to Run.....11
A.3 Complete Source Code.....11

List of Figures

Figure 2.1 Class distribution of the balanced training set.....

Figure 4.1 Training and validation loss curve.....

Figure 4.2 Confusion matrix on the test set.....

Figure 5.1 Screenshot of the application.....

List of Tables

Table 2.1 Dataset summary.....

Table 3.1 Model configuration.....

Table 3.2 Training settings.....

Table 4.1 Results before and after training.....

Table 4.2 Project-specific tasks and results.....

Table 4.3 Error analysis.....

Table 5.1 Sample inputs and outputs of the application.....

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

Project MP-C38 – Contract Clause Type Identifier. A legal contract is a list of clauses such as governing law, notices, counterparts or severability. Lawyers, students and small businesses must find and label these clauses when they review agreements. This project builds an application where the user pastes a clause (or uploads a contract) and the fine-tuned GPT-2 model returns the predicted clause type, written as clause_type(text) → predicted type. The output is only a support tool and must be reviewed by a qualified person.

1.2 Objectives

- Prepare a balanced LEDGAR subset of the 10 most frequent clause types (800 examples per class where available) with a stratified 70/15/15 split.
- Fine-tune GPT-2 (124M) with a new 10-class head and report test accuracy and macro-F1 before and after fine-tuning, with a confusion matrix.
- Provide a clause_type(text) function and a Gradio application that also classifies every clause of an uploaded PDF/TXT contract.

1.3 Category and Approach

Category MP-C: classification fine-tuning of GPT-2 with a new output head (Chapter 6 of the course). The pretrained GPT-2 small model (124M parameters) is loaded into the course-style GPTModel. The language-model head (768 → 50257) is replaced by a new Linear(768 → 10) head. Most weights are frozen; the last transformer blocks, the final layer norm and the head are trained. Macro-F1 is the principal metric because the classes must be treated equally.

CHAPTER 2

DATASET AND PREPROCESSING

2.1 Dataset Description

LEDGAR was created by Tuggener et al. (2020) from contracts filed on the US SEC EDGAR system; each example is one contract provision (text) with a clause-type label. It is included in LexGLUE (Chalkidis et al., 2022) with 100 labels and 60,000 / 10,000 / 10,000 train / validation / test examples. Two sample records: PENDING (print with: python -c "import pandas as pd; print(pd.read_csv('data/train.csv').sample(2, random_state=1))").

Table 2.1: Dataset summary

Item	Details
Dataset name and source (link)	LEDGAR in LexGLUE (config 'ledgar'): https://huggingface.co/datasets/coastalcph/lex_glue
Licence / terms of use	See the dataset card (not verified in this report); used for academic work only
Original size	80,000 provisions, 100 labels (60k/10k/10k) per LexGLUE
Subset used and how it was sampled	Top-10 labels, 800 random examples per class, seed 42 (counts PENDING: run data_preparation.py)
Labels / input-output format	Input: clause text. Output: one of 10 clause types (class names: PENDING)
Train / validation / test sizes	5,600 / 1,200 / 1,200 if 800 per class are available (PENDING confirmation)
Maximum token length used	256 tokens (longer clauses truncated, shorter padded with 50256)

[PENDING: run train.py to create class_distribution.png]

Figure 2.1: Class distribution of the balanced training set

2.2 Preprocessing Steps

- Pool the LEDGAR train, validation and test splits; remove very short texts (< 20 characters) and collapse whitespace.
- Remove duplicates (case, digits and punctuation ignored) before splitting so no clause appears in two splits; choose the 10 most frequent labels; sample the same number of clauses per class with seed 42; stratified 70/15/15 split.
- Counts before/after each step are saved in data/data_info.json. Only exact (normalised) duplicates are removed; near-duplicates with different wording may remain.

2.3 Tokenization and Data Loader

Text is converted to token IDs with the GPT-2 byte-pair-encoding tokenizer (tiktoken 'gpt2'). ClauseDataset truncates each clause to 256 tokens and pads shorter ones with token 50256 (<|endoftext|>) so a batch has one length; it also stores the real length. The DataLoader (batch size 16, shuffled with a fixed seed for training) returns token IDs, lengths and labels. The same encode/pad functions are used in inference.py so the application sees exactly what the model saw in training.

CHAPTER 3
MODEL AND TRAINING SETUP

3.1 Model Architecture

The course GPTModel was re-implemented in gpt_model.py: token and positional embeddings, 12 transformer blocks (causal multi-head self-attention, feed-forward network with GELU, layer norm, residual connections), a final layer norm and an output head. The GPT-2 small (124M) pretrained weights are downloaded as a safetensors file and copied into this model; no Hugging Face model class (AutoModel, Trainer, pipeline) is used for the core model.

Table 3.1: Model configuration

Parameter	Value
Base model	GPT-2 small (124M), pretrained weights from openai-community/gpt2
vocab_size	50257
context_length	1024 (clauses truncated to 256 tokens)
emb_dim	768
n_heads	12
n_layers	12
drop_rate	0.1
qkv_bias	True
Output head	Linear(768 → 10), new and randomly initialised
Trainable layers	last 2 transformer blocks + final LayerNorm + output head
Total / trainable parameters	PENDING (run train.py)

3.2 Changes Made for This Application

The language-model head is replaced by a new Linear(768 → 10) head. All pretrained weights are first frozen; then the last transformer blocks, the final layer norm and the new head are unfrozen, so that training is fast and the general language knowledge of GPT-2 is kept. Because GPT-2 is a causal model, only the last token has seen the whole clause, so the

output at the last real (non-padding) token is used as the clause representation. Padding on the right never affects earlier tokens because of the causal mask.

3.3 Training Settings

Table 3.2: Training settings

Setting	Value
Optimizer	AdamW (cross-entropy loss)
Learning rate / weight decay	5e-05 / 0.1
Batch size	16
Number of epochs	4
Random seed	42 (torch, numpy, random)
Hardware	PENDING (Colab T4 GPU recommended)
Training time	PENDING (run train.py)

CHAPTER 4

RESULTS AND EVALUATION

4.1 Before and After Training

Table 4.1: Results before and after training

Metric	Before training	After training
Test accuracy	PENDING (run train.py)	PENDING (run train.py)
Macro-F1 (principal metric)	PENDING (run train.py)	PENDING (run train.py)
Test loss	PENDING (run train.py)	PENDING (run train.py)

[PENDING: run train.py to create loss_curve.png]

Figure 4.1: Training and validation loss curve

PENDING: describe the curve after running train.py.

4.2 Project-Specific Tasks

Table 4.2: Project-specific tasks and results

Task (from statement)	What I did	Result
(1) Balanced dataset: 10 most frequent clause types, 800 per class	data_preparation.py with stratified 70/15/15 split	PENDING (run train.py)
(2) clause_type(text) → predicted clause type	ClauseClassifier.clause_type() in inference.py, used by the app	Implemented; outputs PENDING until the model is trained
(3) Report test macro-F1	Computed with scikit-learn on test predictions	Macro-F1 = PENDING (run train.py); macro-precision = PENDING (run train.py); macro-recall = PENDING (run train.py)

[PENDING: run train.py to create confusion_matrix.png]

Figure 4.2: Confusion matrix on the test set (per-class precision/recall/F1: report_assets/test_per_class.csv)

4.3 Error Analysis

Table 4.3: Error analysis

Input	Expected	Model output	Likely reason
PENDING	PENDING	PENDING	PENDING
PENDING	PENDING	PENDING	PENDING
PENDING	PENDING	PENDING	PENDING

CHAPTER 5
APPLICATION

5.1 Application Interface

The Gradio application (app.py) loads the saved checkpoint models/best.pt once at start-up; it never retrains. Tab 1: the user types a clause and presses Predict to see the predicted type, the model confidence (softmax probability, not legal certainty) and the top three classes. Tab 2: the user uploads a PDF or TXT contract; the text is extracted, split into clauses (numbering, headings, paragraphs), each clause is classified, and a table plus a downloadable CSV is shown. Clear error messages are given for empty input, unsupported files and extraction failures. A legal disclaimer is displayed.

[PENDING: after running app.py, save a real screenshot as report_assets/app_screenshot.png]

Figure 5.1: Screenshot of the application

5.2 Sample Inputs and Outputs

Table 5.1: Sample inputs and outputs of the application

#	Input	Output	Correct?
1	This Agreement shall be governed by the laws of the State of Delaware.	PENDING	PENDING
2	Any notice under this Agreement shall be in writing and sent by registered mail to the address of the party.	PENDING	PENDING
3	This Agreement may be executed in two or more counterparts, each of which is an original.	PENDING	PENDING
4	If any provision is held unenforceable, the remaining provisions remain in	PENDING	PENDING

#	Input	Output	Correct?
	effect.		
5	No waiver of any breach shall be deemed a waiver of any later breach.	PENDING	PENDING

CHAPTER 6
CONCLUSION AND LIMITATIONS

6.1 Conclusion

The complete pipeline (data preparation, GPT-2 classifier, training, evaluation, application) is implemented. Final numbers are PENDING until train.py is run; update this paragraph with the real accuracy and macro-F1.

6.2 Limitations and Ethical Considerations

- Only the 10 most frequent clause types are supported; other clause types will be forced into one of these 10 classes.
- Clauses are cut to 256 tokens, the model is small (124M), LEDGAR is US-English contract text, and similar classes (e.g. waiver vs. amendment) can be confused. Predictions may be wrong and are not legal advice.
- AI-tool declaration: an AI coding assistant (Claude) was used to generate the project code and draft this report; the student is responsible for understanding and explaining all code.

6.3 Future Enhancements

- Use GPT-2 medium or unfreeze more layers, and train on more LEDGAR classes with more data.
- Add OCR for scanned PDFs, a better clause splitter and show the most similar training clauses as an explanation.

REFERENCES

[1] [Author(s)], *Learn from Scratch: Large Language Models*, [Publisher], [Year]. (Course textbook)

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI Technical Report, 2019.

[3] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] M. Tuggener, P. von Däniken, T. Peetz and M. Cieliebak, “LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts,” in Proc. 12th Language Resources and Evaluation Conference (LREC), 2020, pp. 1235–1241.

[5] I. Chalkidis et al., “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English,” in Proc. 60th Annual Meeting of the ACL, 2022. Available: https://huggingface.co/datasets/coastalcph/lex_glue

[6] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[7] A. Abid et al., “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” arXiv:1906.02569, 2019.

APPENDIX A — CODE, NOTEBOOK LINK AND COMPLETE SOURCE CODE

A.1 Links (both required)

Complete code repository (GitHub / Google Drive folder): *[URL – must contain all .py / .ipynb files, README and requirements.txt]*

Notebook link (Google Colab / Kaggle): *[URL – set sharing to 'Anyone with the link can view' and keep all cell outputs visible]*

Main file: train.py (project folder MP-C38); notebook: MP-C38_clause_classifier.ipynb

Python 3.10+, torch>=2.1, tiktoken, datasets==2.21.0, scikit-learn, gradio (full list: requirements.txt; exact versions are printed by pip in Colab)

A.2 Steps to Run

- pip install -r requirements.txt
- python data_preparation.py (downloads LEDGAR and creates data/*.csv)
- python train.py (trains, saves models/best.pt, results and charts in report_assets/); evaluate again with: python evaluate.py
- python app.py (opens the Gradio application; tests: pytest tests)

A.3 Complete Source Code

A.3.1 utils.py

```
"""Shared helpers: seed, device, tokenizer, Dataset, text extraction, clause
splitting, CSV export."""
import io, re, random
import numpy as np
import pandas as pd
import tiktoken
import torch
from torch.utils.data import Dataset

PAD_ID = 50256                      # GPT-2 <|endoftext|> token, used for
padding (course Ch 6)
TOKENIZER = tiktoken.get_encoding("gpt2")


def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def get_device():
```

```
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def encode(text, max_length):
    """Text -> token ids, truncated to max_length. Same function is used in
training AND inference."""
    return TOKENIZER.encode(str(text), allowed_special=set())[:max_length]


class ClauseDataset(Dataset):
    """Course-style Dataset: tokenizes, truncates, pads every clause to the same
length with PAD_ID.
    Also returns the true (unpadded) length so the model can read the last REAL
token."""
    def __init__(self, texts, labels, max_length=256):
        self.encoded = [encode(t, max_length) or [PAD_ID] for t in texts]
        self.labels = list(labels)
        self.max_length = max(len(e) for e in self.encoded)
        self.max_length = min(self.max_length, max_length)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, i):
        ids = self.encoded[i]
        padded = ids + [PAD_ID] * (self.max_length - len(ids))
        return torch.tensor(padded), torch.tensor(len(ids)),
torch.tensor(self.labels[i])


def pad_batch(token_lists):
    """Pad a list of token-id lists to one tensor (used at inference time)."""
    lengths = [max(len(t), 1) for t in token_lists]
    width = max(lengths)
    ids = [(t or [PAD_ID]) + [PAD_ID] * (width - max(len(t), 1)) for t in
token_lists]
    return torch.tensor(ids), torch.tensor(lengths)


# ---------- contract reading and clause splitting ----------
def extract_text(path):
    """Read text from a .txt or .pdf file. Raises ValueError with a readable
message."""
    path = str(path)
    low = path.lower()
    try:
        if low.endswith(".txt"):
            with open(path, "rb") as f:
                return f.read().decode("utf-8", errors="ignore")
        if low.endswith(".pdf"):
            from pypdf import PdfReader
            reader = PdfReader(path)
            text = "\n".join((p.extract_text() or "") for p in reader.pages)
            if not text.strip():
                raise ValueError("No selectable text found in the PDF (it may be
a scanned image).")
            return text
    except ValueError:
        raise
    except Exception as e:
        raise ValueError(f"Could not read the file: {e}")
    raise ValueError("Unsupported file type. Please upload a .pdf or .txt
file.")


# a new clause usually starts with: 1. / 1.1 / (a) / Section 3 / ARTICLE II
_CLAUSE_START = re.compile(
    r"^\s*(?:\d+(?:\.\d+)*[.)]?\s+\S|\(\w{1,4}\)\s+\S|(?:section|article|
```

```
clause)\s+\w+|[A-Z][A-Z \-]{3,40}\.?\s*$)",
    re.IGNORECASE)


def split_into_clauses(text, min_chars=40, max_chars=1500):
    """Split contract text into clauses (original order kept).
    Rules: numbered/lettered headings start a clause; otherwise blank lines
separate paragraphs;
    very short pieces are merged with the next piece; very long pieces are cut
at sentence ends."""
    text = text.replace("\r", "")
    lines = text.split("\n")
    blocks, cur = [], []
    for line in lines:
        if not line.strip():                        # blank line = paragraph
boundary
            if cur: blocks.append(" ".join(cur)); cur = []
            continue
        if _CLAUSE_START.match(line) and cur:      # numbering / heading = new
clause
            blocks.append(" ".join(cur)); cur = []
        cur.append(line.strip())
    if cur: blocks.append(" ".join(cur))
    blocks = [re.sub(r"\s+", " ", b).strip() for b in blocks if b.strip()]

    merged, carry = [], ""                         # merge tiny pieces (e.g. lone
headings) forward
    for b in blocks:
        b = (carry + " " + b).strip() if carry else b
        if len(b) < min_chars: carry = b
        else: merged.append(b); carry = ""
    if carry: merged.append(carry) if not merged else merged.__setitem__(-1,
merged[-1] + " " + carry)

    clauses = []
    for b in merged:                               # cut very long blocks at
sentence boundaries
        if len(b) <= max_chars: clauses.append(b); continue
        piece = ""
        for s in re.split(r"(?<=[.;])\s+", b):
            if len(piece) + len(s) > max_chars and piece:
                clauses.append(piece.strip()); piece = ""
            piece += " " + s
        if piece.strip(): clauses.append(piece.strip())
    return clauses


def results_to_csv(rows):
    """rows: list of dicts -> CSV string (clause order preserved)."""
    buf = io.StringIO()
    pd.DataFrame(rows).to_csv(buf, index=False)
    return buf.getvalue()
```

A.3.2 gpt_model.py

```
"""GPT-2 architecture written from scratch (course Ch 3-4) + loading of
pretrained GPT-2 124M weights (Ch 5)
+ new classification head (Ch 6). No Hugging Face model classes are used; the HF
hub is only used to
download the weight file."""
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

GPT2_SMALL = dict(vocab_size=50257, context_length=1024, emb_dim=768,
n_heads=12,
                  n_layers=12, drop_rate=0.1, qkv_bias=True)
```

```
class MultiHeadAttention(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        d = cfg["emb_dim"]; self.h = cfg["n_heads"]
        self.W_query = nn.Linear(d, d, bias=cfg["qkv_bias"])
        self.W_key = nn.Linear(d, d, bias=cfg["qkv_bias"])
        self.W_value = nn.Linear(d, d, bias=cfg["qkv_bias"])
        self.out_proj = nn.Linear(d, d)
        self.drop = cfg["drop_rate"]

    def forward(self, x):
        b, t, d = x.shape
        split = lambda m: m(x).view(b, t, self.h, d // self.h).transpose(1, 2)
        q, k, v = split(self.W_query), split(self.W_key), split(self.W_value)
        # causal mask: each token only sees earlier tokens (so right-padding
never leaks into real tokens)
        out = F.scaled_dot_product_attention(q, k, v, is_causal=True,
                                             dropout_p=self.drop if
self.training else 0.0)
        return self.out_proj(out.transpose(1, 2).reshape(b, t, d))


class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        d = cfg["emb_dim"]
        self.att = MultiHeadAttention(cfg)
        self.ff = nn.Sequential(nn.Linear(d, 4 * d),
nn.GELU(approximate="tanh"), nn.Linear(4 * d, d))
        self.norm1, self.norm2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.drop = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        x = x + self.drop(self.att(self.norm1(x)))
        return x + self.drop(self.ff(self.norm2(x)))


class GPTModel(nn.Module):
    """GPT-2 backbone. out_dim=None -> language-model head (vocab). out_dim=10 -
> classification head."""
    def __init__(self, cfg, out_dim=None):
        super().__init__()
        self.cfg = cfg
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential(*[TransformerBlock(cfg) for _ in
range(cfg["n_layers"])])
        self.final_norm = nn.LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], out_dim or cfg["vocab_size"],
bias=out_dim is not None)

    def forward(self, ids):                       # ids: [B, T] -> logits [B, T,
out_dim]
        pos = torch.arange(ids.shape[1], device=ids.device)
        x = self.drop_emb(self.tok_emb(ids) + self.pos_emb(pos))
        return self.out_head(self.final_norm(self.trf_blocks(x)))

    def classify_logits(self, ids, lengths):
        """One vector of class scores per clause, read at the last REAL (non-
pad) token."""
        logits = self(ids)
        idx = (lengths - 1).clamp(min=0).to(logits.device)
        return logits[torch.arange(ids.shape[0], device=logits.device), idx]


def load_gpt2_weights(model, repo_id="openai-community/gpt2"):
```

```
    """Download GPT-2 124M weights (safetensors) and copy them into OUR
GPTModel."""
    from huggingface_hub import hf_hub_download
    from safetensors.torch import load_file
    sd = {k.replace("transformer.", ""): v for k, v in
load_file(hf_hub_download(repo_id, "model.safetensors")).items()}
    d = model.cfg["emb_dim"]
    with torch.no_grad():
        model.tok_emb.weight.copy_(sd["wte.weight"]);
model.pos_emb.weight.copy_(sd["wpe.weight"])
        for i, blk in enumerate(model.trf_blocks):
            p = f"h.{i}."
            w, b = sd[p + "attn.c_attn.weight"].T, sd[p + "attn.c_attn.bias"]
# HF stores Conv1D transposed
            for j, lin in enumerate([blk.att.W_query, blk.att.W_key,
blk.att.W_value]):
                lin.weight.copy_(w[j * d:(j + 1) * d]); lin.bias.copy_(b[j * d:
(j + 1) * d])
            blk.att.out_proj.weight.copy_(sd[p + "attn.c_proj.weight"].T);
blk.att.out_proj.bias.copy_(sd[p + "attn.c_proj.bias"])
            blk.ff[0].weight.copy_(sd[p + "mlp.c_fc.weight"].T);
blk.ff[0].bias.copy_(sd[p + "mlp.c_fc.bias"])
            blk.ff[2].weight.copy_(sd[p + "mlp.c_proj.weight"].T);
blk.ff[2].bias.copy_(sd[p + "mlp.c_proj.bias"])
            blk.norm1.weight.copy_(sd[p + "ln_1.weight"]);
blk.norm1.bias.copy_(sd[p + "ln_1.bias"])
            blk.norm2.weight.copy_(sd[p + "ln_2.weight"]);
blk.norm2.bias.copy_(sd[p + "ln_2.bias"])
        model.final_norm.weight.copy_(sd["ln_f.weight"]);
model.final_norm.bias.copy_(sd["ln_f.bias"])
    return model


def build_classifier(num_classes, cfg=GPT2_SMALL, pretrained=True,
unfreeze_blocks=1):
    """Pretrained GPT-2 + NEW trainable head. Freeze everything, then unfreeze
the last
    `unfreeze_blocks` transformer blocks, the final LayerNorm and the head
(course Ch 6)."""
    model = GPTModel(cfg)
    if pretrained:
        load_gpt2_weights(model)
    model.out_head = nn.Linear(cfg["emb_dim"], num_classes)       # new head,
random init
    for p in model.parameters():
        p.requires_grad = False
    for blk in list(model.trf_blocks)[len(model.trf_blocks) - unfreeze_blocks:]:
        for p in blk.parameters(): p.requires_grad = True
    for p in list(model.final_norm.parameters()) +
list(model.out_head.parameters()):
        p.requires_grad = True
    return model


def count_params(model):
    total = sum(p.numel() for p in model.parameters())
    return total, sum(p.numel() for p in model.parameters() if p.requires_grad)
```

A.3.3 data_preparation.py

```
"""Prepare the balanced 10-class LEDGAR subset (LexGLUE) and split 70/15/15."""
import json, re, argparse, os
import pandas as pd
from sklearn.model_selection import train_test_split


def normalise_key(text):
    """Key for duplicate detection: lowercase, letters only (ignores
digits/punctuation/spacing)."""
```

```
    return re.sub(r"[^a-z]+", "", str(text).lower())


def build_balanced(df, n_classes=10, per_class=800, seed=42):
    """df has columns text,label_name. Returns (balanced_df, info dict). Pure
pandas -> easy to test."""
    df = df.copy()
    df["text"] = df["text"].astype(str).str.replace(r"\s+", " ",
regex=True).str.strip()
    df = df[df["text"].str.len() >= 20]
    df["key"] = df["text"].map(normalise_key)
    n_before = len(df)
    df = df.drop_duplicates("key")                        # removes duplicates
BEFORE splitting -> no leakage
    # a text that appears under two different labels is ambiguous -> already
collapsed by drop_duplicates
    counts = df["label_name"].value_counts()
    top = list(counts.index[:n_classes])
    n_use = int(min(per_class, counts[top].min()))
    parts = [df[df.label_name == c].sample(n=n_use, random_state=seed) for c in
top]
    bal = pd.concat(parts).reset_index(drop=True)
    label_map = {c: i for i, c in enumerate(sorted(top))}
    bal["label"] = bal["label_name"].map(label_map)
    info = dict(examples_before_dedup=n_before, examples_after_dedup=len(df),
                top_classes_counts={c: int(counts[c]) for c in top},
per_class_used=n_use,
                shortfall=n_use < per_class, label_map=label_map, seed=seed)
    return bal[["text", "label", "label_name"]], info


def split_data(bal, seed=42):
    train, rest = train_test_split(bal, test_size=0.30, stratify=bal["label"],
random_state=seed)
    val, test = train_test_split(rest, test_size=0.50, stratify=rest["label"],
random_state=seed)
    return train, val, test


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default="data"); ap.add_argument("--seed",
type=int, default=42)
    ap.add_argument("--per_class", type=int, default=800)
    a = ap.parse_args()
    import datasets
    try:
        ds = datasets.load_dataset("coastalcph/lex_glue", "ledgar",
trust_remote_code=True)
    except Exception:
        ds = datasets.load_dataset("coastalcph/lex_glue", "ledgar",
revision="refs/convert/parquet")
    print(ds)                                             # inspect splits /
fields
    names = ds["train"].features["label"].names
    pool = pd.concat([ds[s].to_pandas() for s in ds.keys()])   # pool all
splits, we re-split ourselves
    pool["label_name"] = pool["label"].map(lambda i: names[i])
    bal, info = build_balanced(pool[["text", "label_name"]],
per_class=a.per_class, seed=a.seed)
    train, val, test = split_data(bal, a.seed)
    os.makedirs(a.out, exist_ok=True)
    for n, d in [("train", train), ("val", val), ("test", test)]:
        d.to_csv(f"{a.out}/{n}.csv", index=False)
    info.update(original_splits={s: len(ds[s]) for s in ds.keys()},
original_num_labels=len(names),
                split_sizes=dict(train=len(train), val=len(val),
test=len(test)),
```

```
                source="https://huggingface.co/datasets/coastalcph/lex_glue
(config: ledgar)",
                datasets_version=datasets.__version__,
                selection_rule="10 most frequent labels after pooling all splits
and removing duplicates")
    json.dump(info, open(f"{a.out}/data_info.json", "w"), indent=2)
    json.dump(info["label_map"], open(f"{a.out}/label_map.json", "w"), indent=2)
    bal["label_name"].value_counts().to_csv(f"{a.out}/class_distribution.csv")
    print(json.dumps(info, indent=2))


if __name__ == "__main__":
    main()
```

A.3.4 train.py

```
"""Fine-tune GPT-2 124M with a new 10-class head. Saves best checkpoint (by
validation macro-F1)."""
import json, time, argparse, os
import pandas as pd, torch, torch.nn.functional as F
from torch.utils.data import DataLoader
from gpt_model import build_classifier, count_params, GPT2_SMALL
from utils import ClauseDataset, set_seed, get_device
from evaluate import run_eval, metrics, save_report


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", default="data"); ap.add_argument("--epochs",
type=int, default=4)
    ap.add_argument("--batch", type=int, default=16); ap.add_argument("--lr",
type=float, default=5e-5)
    ap.add_argument("--wd", type=float, default=0.1); ap.add_argument("--
max_len", type=int, default=256)
    ap.add_argument("--unfreeze", type=int, default=2, help="number of last
transformer blocks to unfreeze")
    ap.add_argument("--seed", type=int, default=42); ap.add_argument("--out",
default="models/best.pt")
    a = ap.parse_args()
    set_seed(a.seed); dev = get_device(); os.makedirs("models", exist_ok=True);
os.makedirs("report_assets", exist_ok=True)
    label_map = json.load(open(f"{a.data}/label_map.json")); names = [k for k, _
in sorted(label_map.items(), key=lambda x: x[1])]
    tr, va, te = [pd.read_csv(f"{a.data}/{s}.csv") for s in ("train", "val",
"test")]
    mk = lambda d, sh: DataLoader(ClauseDataset(d.text, d.label, a.max_len),
batch_size=a.batch, shuffle=sh)
    g = torch.Generator().manual_seed(a.seed)
    tl = DataLoader(ClauseDataset(tr.text, tr.label, a.max_len),
batch_size=a.batch, shuffle=True, generator=g)
    vl, sl = mk(va, False), mk(te, False)

    model = build_classifier(len(names), GPT2_SMALL, pretrained=True,
unfreeze_blocks=a.unfreeze).to(dev)
    total, trainable = count_params(model); print(f"params total={total:,}
trainable={trainable:,} device={dev}")
    res = dict(total_params=total, trainable_params=trainable, device=str(dev),
seed=a.seed, settings=vars(a))

    # accuracy BEFORE fine-tuning (pretrained GPT-2 + untrained random head) on
the test set
    p, y, l = run_eval(model, sl, dev); res["before_test"] = {**metrics(y, p),
"test_loss": l}; print("before:", res["before_test"])

    opt = torch.optim.AdamW([q for q in model.parameters() if q.requires_grad],
lr=a.lr, weight_decay=a.wd)
    hist, best, t0 = [], -1, time.time()
    for ep in range(1, a.epochs + 1):
        model.train(); run, n, correct = 0.0, 0, 0
```

```
        for ids, lengths, y in tl:
            ids, y = ids.to(dev), y.to(dev)
            logits = model.classify_logits(ids, lengths)
            loss = F.cross_entropy(logits, y)
            opt.zero_grad(); loss.backward(); opt.step()
            run += loss.item() * len(y); n += len(y); correct +=
(logits.argmax(-1) == y).sum().item()
        vp, vy, vloss = run_eval(model, vl, dev); vm = metrics(vy, vp)
        hist.append(dict(epoch=ep, train_loss=run / n, train_acc=correct / n,
val_loss=vloss, val_acc=vm["accuracy"], val_macro_f1=vm["macro_f1"]))
        print(hist[-1])
        if vm["macro_f1"] > best:
            best = vm["macro_f1"]
            torch.save(dict(state_dict=model.state_dict(), cfg=GPT2_SMALL,
class_names=names, label_map=label_map,
                            max_length=a.max_len, seed=a.seed), a.out)
    res["train_seconds"] = time.time() - t0; res["history"] = hist;
res["best_val_macro_f1"] = best

    # reload best checkpoint and evaluate ONCE on the held-out test set
    ck = torch.load(a.out, map_location=dev);
model.load_state_dict(ck["state_dict"])
    p, y, l = run_eval(model, sl, dev); res["after_test"] = {**metrics(y, p),
"test_loss": l}; print("after:", res["after_test"])
    save_report(y, p, names, te.text.tolist())
    json.dump(res, open("report_assets/results.json", "w"), indent=2)

    import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
    ep = [h["epoch"] for h in hist]; plt.figure(figsize=(6, 4))
    plt.plot(ep, [h["train_loss"] for h in hist], "o-", label="train");
plt.plot(ep, [h["val_loss"] for h in hist], "s-", label="validation")
    plt.xlabel("Epoch"); plt.ylabel("Cross-entropy loss"); plt.legend();
plt.title("Training and validation loss"); plt.tight_layout()
    plt.savefig("report_assets/loss_curve.png", dpi=150)
    tr["label_name"].value_counts().plot(kind="barh", figsize=(7, 4),
title="Training-set class distribution"); plt.tight_layout()
    plt.savefig("report_assets/class_distribution.png", dpi=150)


if __name__ == "__main__":
    main()
```

A.3.5 evaluate.py

```
"""Evaluation: loss/accuracy/macro metrics, per-class report, confusion matrix,
error examples."""
import json, argparse
import numpy as np, pandas as pd, torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from sklearn.metrics import accuracy_score, precision_recall_fscore_support,
confusion_matrix, classification_report


@torch.no_grad()
def run_eval(model, loader, device):
    model.eval(); preds, labels, tot = [], [], 0.0
    for ids, lengths, y in loader:
        logits = model.classify_logits(ids.to(device), lengths)
        tot += F.cross_entropy(logits, y.to(device), reduction="sum").item()
        preds += logits.argmax(-1).cpu().tolist(); labels += y.tolist()
    return np.array(preds), np.array(labels), tot / len(labels)


def metrics(labels, preds):
    p, r, f, _ = precision_recall_fscore_support(labels, preds, average="macro",
zero_division=0)
    return dict(accuracy=accuracy_score(labels, preds), macro_precision=p,
macro_recall=r, macro_f1=f)
```

```
def save_report(labels, preds, names, texts, out="report_assets", tag="test"):
    import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
    rep = classification_report(labels, preds, target_names=names,
output_dict=True, zero_division=0)
    pd.DataFrame(rep).T.to_csv(f"{out}/{tag}_per_class.csv")
    cm = confusion_matrix(labels, preds); pd.DataFrame(cm, index=names,
columns=names).to_csv(f"{out}/{tag}_confusion.csv")
    fig, ax = plt.subplots(figsize=(8, 7)); ax.imshow(cm, cmap="Blues")
    ax.set_xticks(range(len(names))); ax.set_yticks(range(len(names)))
    ax.set_xticklabels(names, rotation=60, ha="right", fontsize=7);
ax.set_yticklabels(names, fontsize=7)
    for i in range(len(names)):
        for j in range(len(names)): ax.text(j, i, cm[i, j], ha="center",
va="center", fontsize=7)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title("Confusion
matrix (test set)")
    fig.tight_layout(); fig.savefig(f"{out}/confusion_matrix.png", dpi=150);
plt.close(fig)
    wrong = [dict(text=t[:300], expected=names[l], predicted=names[p]) for t, l,
p in zip(texts, labels, preds) if l != p]
    pd.DataFrame(wrong).to_csv(f"{out}/errors.csv", index=False)


def main():
    from inference import ClauseClassifier
    from utils import ClauseDataset, get_device
    ap = argparse.ArgumentParser(); ap.add_argument("--ckpt",
default="models/best.pt"); ap.add_argument("--data", default="data")
    a = ap.parse_args()
    clf = ClauseClassifier(a.ckpt); test = pd.read_csv(f"{a.data}/test.csv")
    dl = DataLoader(ClauseDataset(test.text, test.label, clf.max_length),
batch_size=32)
    preds, labels, loss = run_eval(clf.model, dl, clf.device)
    m = metrics(labels, preds); m["test_loss"] = loss
    save_report(labels, preds, clf.class_names, test.text.tolist())
    prev = json.load(open("report_assets/results.json")) if
__import__("os").path.exists("report_assets/results.json") else {}
    prev["after_test"] = m; json.dump(prev, open("report_assets/results.json",
"w"), indent=2); print(m)


if __name__ == "__main__":
    main()
```

A.3.6 inference.py

```
"""Loads the saved checkpoint once and exposes clause_type(text) / batch
prediction.
Uses the same tokenizer, truncation, padding and label map as training."""
import torch
from gpt_model import GPTModel
from utils import encode, pad_batch, get_device


class ClauseClassifier:
    def __init__(self, ckpt_path="models/best.pt"):
        self.device = get_device()
        ck = torch.load(ckpt_path, map_location=self.device)
        self.class_names, self.max_length = ck["class_names"], ck["max_length"]
        self.model = GPTModel(ck["cfg"], out_dim=len(self.class_names))
        self.model.load_state_dict(ck["state_dict"]);
self.model.to(self.device).eval()

    @torch.no_grad()
    def predict_batch(self, texts, top_k=3, batch_size=16):
        out = []
        for i in range(0, len(texts), batch_size):
```

```
            ids, lengths = pad_batch([encode(t, self.max_length) for t in
texts[i:i + batch_size]])
            probs =
torch.softmax(self.model.classify_logits(ids.to(self.device), lengths), dim=-
1).cpu()
            for p in probs:
                v, ix = p.topk(min(top_k, len(self.class_names)))
                out.append([(self.class_names[j], float(s)) for s, j in zip(v,
ix)])
        return out                                  # per clause: [(label,
prob), ...] best first

    def clause_type(self, text):
        """Main function: clause text -> predicted clause type."""
        return self.predict_batch([text])[0][0][0]
```

A.3.7 app.py

```
"""Gradio app: single-clause classification + contract upload. Loads
models/best.pt (no retraining)."""
import os, tempfile
import gradio as gr
from inference import ClauseClassifier
from utils import extract_text, split_into_clauses, results_to_csv

CKPT = os.environ.get("CKPT", "models/best.pt")
clf = ClauseClassifier(CKPT) if os.path.exists(CKPT) else None
DISCLAIMER = ("**Academic NLP prototype - not legal advice.** Predictions may be
wrong and must be reviewed by a "
              "qualified person. Confidence is the model's softmax probability,
not legal certainty.")
EXAMPLES = ["This Agreement shall be governed by and construed in accordance
with the laws of the State of New York.",
            "This Agreement may be executed in counterparts, each of which shall
be deemed an original.",
            "If any provision of this Agreement is held invalid, the remaining
provisions shall remain in full force."]


def need_model():
    if clf is None:
        raise gr.Error(f"Model checkpoint '{CKPT}' not found. Run train.py
first.")


def predict_single(text):
    need_model()
    if not text or not text.strip():
        raise gr.Error("Please enter a clause first.")
    top = clf.predict_batch([text], top_k=3)[0]
    return top[0][0], f"{top[0][1]:.1%} (model confidence)", {k: v for k, v in
top}


def predict_contract(file):
    need_model()
    if file is None:
        raise gr.Error("Please upload a PDF or TXT contract.")
    try:
        clauses = split_into_clauses(extract_text(file if isinstance(file, str)
else file.name))
    except ValueError as e:
        raise gr.Error(str(e))
    if not clauses:
        raise gr.Error("No clauses could be extracted from this file.")
    preds = clf.predict_batch(clauses, top_k=1)
    rows = [dict(clause_no=i + 1, clause_text=c, predicted_type=p[0][0],
confidence=round(p[0][1], 4))
            for i, (c, p) in enumerate(zip(clauses, preds))]
```

```
    path = os.path.join(tempfile.mkdtemp(), "clause_predictions.csv")
    open(path, "w", encoding="utf-8").write(results_to_csv(rows))
    table = [[r["clause_no"], r["clause_text"][:300], r["predicted_type"],
f"{r['confidence']:.1%}"] for r in rows]
    return table, path


with gr.Blocks(title="Contract Clause Type Identifier") as demo:
    gr.Markdown("# Contract Clause Type Identifier\nGPT-2 (124M) fine-tuned on
10 LEDGAR clause types. " + DISCLAIMER)
    gr.Markdown("**Supported clause types:** " + (", ".join(clf.class_names) if
clf else "(train the model first)"))
    with gr.Tab("Single clause"):
        box = gr.Textbox(lines=6, label="Contract clause")
        gr.Examples(EXAMPLES, box, label="Illustrative examples (predictions
still come from the trained model)")
        btn = gr.Button("Predict", variant="primary")
        lab, conf, top3 = gr.Textbox(label="Predicted clause type"),
gr.Textbox(label="Confidence"), gr.Label(label="Top 3")
        btn.click(predict_single, box, [lab, conf, top3])
    with gr.Tab("Upload contract"):
        up = gr.File(label="PDF or TXT", file_types=[".pdf", ".txt"])
        go = gr.Button("Classify all clauses", variant="primary")
        tbl = gr.Dataframe(headers=["#", "Clause", "Predicted type",
"Confidence"], wrap=True)
        dl = gr.File(label="Download CSV")
        go.click(predict_contract, up, [tbl, dl])

if __name__ == "__main__":
    demo.launch()
```

A.3.8 fill_report.py

```
"""Fill the college report template. Re-run after training to insert real
numbers and figures:
    python fill_report.py LLM_Mini_project.docx MP-C38_Report.docx
Anything not yet produced is marked PENDING (never invented)."""
import sys, os, json, copy, glob, re
import pandas as pd
from docx import Document
from docx.shared import Pt, RGBColor, Inches

src, dst = sys.argv[1], sys.argv[2]
d = Document(src); P = list(d.paragraphs); T = d.tables
J = lambda p: json.load(open(p)) if os.path.exists(p) else {}
R, DI = J("report_assets/results.json"), J("data/data_info.json")
PEND = "PENDING (run train.py)"
g = lambda *k: (lambda x: x if x is not None else PEND)
(__import__("functools").reduce(lambda a, b: a.get(b) if isinstance(a, dict)
else None, k, R))
fx = lambda v: f"{v:.4f}" if isinstance(v, float) else str(v)
a_, b_ = R.get("after_test", {}), R.get("before_test", {})
N = lambda m, k: fx(m[k]) if k in m else PEND
TITLE, BLACK = "Contract Clause Type Identifier", RGBColor(0, 0, 0)


def settext(p, text, size=None):
    for r in p.runs[1:]: r._element.getparent().remove(r._element)
    r = p.runs[0] if p.runs else p.add_run(); r.text = text
    r.font.italic = False; r.font.color.rgb = BLACK
    if size: r.font.size = size


def cell(c, text): settext(c.paragraphs[0], text) if c.paragraphs[0].runs else
c.paragraphs[0].add_run(text)
def image_or_pending(c, path, note):
    p = c.paragraphs[0]
    for r in p.runs: r._element.getparent().remove(r._element)
    if os.path.exists(path): p.add_run().add_picture(path, width=Inches(5.3))
```

```
    else: p.add_run(f"[PENDING: {note}]")


# ---- title page / certificate ----
settext(P[6], TITLE.upper(), Pt(18)); settext(P[7], "Project ID: MP-C38")
for i, t in enumerate(["SUHANI S RAO", "1GA23IS166", "7B"]):
cell(T[0].rows[0].cells[i], t)
for r in P[28].runs:
    r.text = (r.text.replace("[Title of the Mini Project]", TITLE).replace("[MP-
XXX]", "MP-C38")
              .replace("[STUDENT NAME]", "SUHANI S RAO").replace("[USN]",
"1GA23IS166").replace("[7A/7B]", "7B"))
cell(T[1].rows[0].cells[0], "SDG 16: Peace, Justice and Strong Institutions    |
SDG 8: Decent Work and Economic Growth")

# ---- abstract ----
f1 = N(a_, "macro_f1")
res_s = (f"On the held-out test set the model reached an accuracy of {N(a_,
'accuracy')} and a macro-F1 of {f1}, compared with "
         f"{N(b_, 'accuracy')} and {N(b_, 'macro_f1')} before fine-tuning.") if
a_ else "Final test accuracy and macro-F1 are PENDING until train.py is run."
n_cls = DI.get("per_class_used", "800")
settext(P[34], "Legal contracts contain many clauses, and manually labelling
each clause (for example governing law, notices or counterparts) "
 "is slow. This project built a Contract Clause Type Identifier that predicts
the type of a clause from its text. The LEDGAR dataset from the "
 f"LexGLUE benchmark was used. The 10 most frequent clause types were selected
and a balanced subset of {n_cls} clauses per class was prepared "
 "(70% training, 15% validation, 15% test, random seed 42). The GPT-2 124M model
from the course was re-implemented, pretrained weights were loaded, and a new "
 "10-class linear classification head was added. The head, the final layer norm
and the last transformer blocks were fine-tuned with AdamW and cross-entropy
loss, "
 f"and the best checkpoint was chosen by validation macro-F1. {res_s} A Gradio
application classifies a single clause or every clause of an uploaded PDF/TXT
contract "
 "and exports the results as CSV. The system is an academic prototype and not
legal advice.")
for r in P[36].runs[1:]: r._element.getparent().remove(r._element)
P[36].add_run("GPT-2, fine-tuning, clause classification, LEDGAR, legal NLP,
Gradio")

# ---- lists of figures/tables ----
settext(P[39], "")
for i in range(40, 52): P[i].runs[-1].text = P[i].runs[-1].text.replace("[  ]",
"")
settext(P[40], "Figure 2.1   Class distribution of the balanced training set\
t"); settext(P[42], "Figure 4.2   Confusion matrix on the test set\t")
settext(P[43], "Figure 5.1   Screenshot of the application\t")

# ---- Chapter 1 ----
settext(P[57], "Project MP-C38 – Contract Clause Type Identifier. A legal
contract is a list of clauses such as governing law, notices, counterparts or
severability. "
 "Lawyers, students and small businesses must find and label these clauses when
they review agreements. This project builds an application where the user pastes
a clause "
 "(or uploads a contract) and the fine-tuned GPT-2 model returns the predicted
clause type, written as clause_type(text) → predicted type. The output is only a
support tool "
 "and must be reviewed by a qualified person.")
for i, t in zip((60, 61, 62), ["Prepare a balanced LEDGAR subset of the 10 most
frequent clause types (800 examples per class where available) with a stratified
70/15/15 split.",
                               "Fine-tune GPT-2 (124M) with a new 10-class head
and report test accuracy and macro-F1 before and after fine-tuning, with a
confusion matrix.",
                               "Provide a clause_type(text) function and a
```

```
Gradio application that also classifies every clause of an uploaded PDF/TXT
contract."]): settext(P[i], t)
settext(P[65], "Category MP-C: classification fine-tuning of GPT-2 with a new
output head (Chapter 6 of the course). The pretrained GPT-2 small model (124M
parameters) is loaded into "
 "the course-style GPTModel. The language-model head (768 → 50257) is replaced
by a new Linear(768 → 10) head. Most weights are frozen; the last transformer
blocks, the final "
 "layer norm and the head are trained. Macro-F1 is the principal metric because
the classes must be treated equally.")

# ---- Chapter 2 ----
settext(P[69], "LEDGAR was created by Tuggener et al. (2020) from contracts
filed on the US SEC EDGAR system; each example is one contract provision (text)
with a "
 "clause-type label. It is included in LexGLUE (Chalkidis et al., 2022) with 100
labels and 60,000 / 10,000 / 10,000 train / validation / test examples. "
 "Two sample records: PENDING (print with: python -c \"import pandas as pd;
print(pd.read_csv('data/train.csv').sample(2, random_state=1))\").")
ds = [["Dataset name and source (link)", "LEDGAR in LexGLUE (config 'ledgar'):
https://huggingface.co/datasets/coastalcph/lex_glue"],
      ["Licence / terms of use", "See the dataset card (not verified in this
report); used for academic work only"],
      ["Original size", "80,000 provisions, 100 labels (60k/10k/10k) per
LexGLUE" + (f"; measured: {DI['original_splits']}" if DI else "")],
      ["Subset used and how it was sampled", (f"All splits pooled, duplicates
removed ({DI['examples_before_dedup']} → {DI['examples_after_dedup']}), top-10
labels, "
        f"{DI['per_class_used']} random examples per class, seed {DI['seed']}")
if DI else "Top-10 labels, 800 random examples per class, seed 42 (counts
PENDING: run data_preparation.py)"],
      ["Labels / input-output format", "Input: clause text. Output: one of 10
clause types (class names: " + (", ".join(DI["label_map"]) if DI else "PENDING")
+ ")"],
      ["Train / validation / test sizes", str(DI["split_sizes"]) if DI else
"5,600 / 1,200 / 1,200 if 800 per class are available (PENDING confirmation)"],
      ["Maximum token length used", "256 tokens (longer clauses truncated,
shorter padded with 50256)"]]
for i, (k, v) in enumerate(ds, 1): cell(T[3].rows[i].cells[1], v)
image_or_pending(T[4].rows[0].cells[0], "report_assets/class_distribution.png",
"run train.py to create class_distribution.png")
settext(P[72], "Figure 2.1: Class distribution of the balanced training set")
for i, t in zip((75, 76), ["Pool the LEDGAR train, validation and test splits;
remove very short texts (< 20 characters) and collapse whitespace.",
                           "Remove duplicates (case, digits and punctuation
ignored) before splitting so no clause appears in two splits; choose the 10 most
frequent labels; sample the same number of clauses per class with seed 42;
stratified 70/15/15 split."]): settext(P[i], t)
I = P[76]
for t in ["Counts before/after each step are saved in data/data_info.json. Only
exact (normalised) duplicates are removed; near-duplicates with different
wording may remain."]:
    n = copy.deepcopy(I._element); I._element.addnext(n)
    from docx.text.paragraph import Paragraph; settext(Paragraph(n, I._parent),
t)
settext(P[79], "Text is converted to token IDs with the GPT-2 byte-pair-encoding
tokenizer (tiktoken 'gpt2'). ClauseDataset truncates each clause to 256 tokens
and pads shorter ones with token 50256 "
 "(<|endoftext|>) so a batch has one length; it also stores the real length. The
DataLoader (batch size 16, shuffled with a fixed seed for training) returns
token IDs, lengths and labels. "
 "The same encode/pad functions are used in inference.py so the application sees
exactly what the model saw in training.")

# ---- Chapter 3 ----
settext(P[83], "The course GPTModel was re-implemented in gpt_model.py: token
and positional embeddings, 12 transformer blocks (causal multi-head self-
attention, feed-forward network with GELU, layer norm, residual connections), "
```

```
 "a final layer norm and an output head. The GPT-2 small (124M) pretrained
weights are downloaded as a safetensors file and copied into this model; no
Hugging Face model class (AutoModel, Trainer, pipeline) is used for the core
model.")
cfg = [("Base model", "GPT-2 small (124M), pretrained weights from openai-
community/gpt2"), ("vocab_size", "50257"), ("context_length", "1024 (clauses
truncated to 256 tokens)"),
       ("emb_dim", "768"), ("n_heads", "12"), ("n_layers", "12"), ("drop_rate",
"0.1"), ("qkv_bias", "True"),
       ("Output head", "Linear(768 → 10), new and randomly initialised"),
("Trainable layers", f"last {R.get('settings', {}).get('unfreeze', 2)}
transformer blocks + final LayerNorm + output head"),
       ("Total / trainable parameters", f"{R['total_params']:,} /
{R['trainable_params']:,}" if R else PEND)]
for i, (k, v) in enumerate(cfg, 1): cell(T[5].rows[i].cells[1], v)
settext(P[87], "The language-model head is replaced by a new Linear(768 → 10)
head. All pretrained weights are first frozen; then the last transformer blocks,
the final layer norm and the new head are unfrozen, "
 "so that training is fast and the general language knowledge of GPT-2 is kept.
Because GPT-2 is a causal model, only the last token has seen the whole clause,
so the output at the last real (non-padding) token is used as the clause
representation. "
 "Padding on the right never affects earlier tokens because of the causal
mask.")
S = R.get("settings", {})
ts = [("Optimizer", "AdamW (cross-entropy loss)"), ("Learning rate / weight
decay", f"{S.get('lr', 5e-5)} / {S.get('wd', 0.1)}"), ("Batch size",
str(S.get("batch", 16))),
      ("Number of epochs", str(S.get("epochs", 4))), ("Random seed",
f"{S.get('seed', 42)} (torch, numpy, random)"),
      ("Hardware", R.get("device", "PENDING (Colab T4 GPU recommended)")),
("Training time", f"{R['train_seconds']:.0f} s" if "train_seconds" in R else
PEND)]
for i, (k, v) in enumerate(ts, 1): cell(T[6].rows[i].cells[1], v)

# ---- Chapter 4 ----
for i, (k, key) in enumerate([("Test accuracy", "accuracy"), ("Macro-F1
(principal metric)", "macro_f1"), ("Test loss", "test_loss")], 1):
    cell(T[7].rows[i].cells[0], k); cell(T[7].rows[i].cells[1], N(b_, key));
cell(T[7].rows[i].cells[2], N(a_, key))
H = R.get("history", [])
settext(P[97], (f"Training loss fell from {H[0]['train_loss']:.3f} to {H[-1]
['train_loss']:.3f} and validation loss went from {H[0]['val_loss']:.3f} to {H[-
1]['val_loss']:.3f} over {len(H)} epochs. "
  f"Best validation macro-F1 was {R['best_val_macro_f1']:.4f}. Overfitting is
indicated if validation loss rises while training loss keeps falling.") if H
else "PENDING: describe the curve after running train.py.")
image_or_pending(T[8].rows[0].cells[0], "report_assets/loss_curve.png", "run
train.py to create loss_curve.png")
tasks = [("(1) Balanced dataset: 10 most frequent clause types, 800 per class",
"data_preparation.py with stratified 70/15/15 split", f"{DI['per_class_used']}
per class, splits {DI['split_sizes']}" if DI else PEND),
         ("(2) clause_type(text) → predicted clause type",
"ClauseClassifier.clause_type() in inference.py, used by the app", "Implemented;
outputs PENDING until the model is trained"),
         ("(3) Report test macro-F1", "Computed with scikit-learn on test
predictions", f"Macro-F1 = {f1}; macro-precision = {N(a_, 'macro_precision')};
macro-recall = {N(a_, 'macro_recall')}")]
for i, row in enumerate(tasks, 1):
    for j, v in enumerate(row): cell(T[9].rows[i].cells[j], v)
image_or_pending(T[10].rows[0].cells[0], "report_assets/confusion_matrix.png",
"run train.py to create confusion_matrix.png")
settext(P[102], "Figure 4.2: Confusion matrix on the test set (per-class
precision/recall/F1: report_assets/test_per_class.csv)")
err = pd.read_csv("report_assets/errors.csv").head(3) if
os.path.exists("report_assets/errors.csv") else None
for i in range(1, 4):
    row = [err.iloc[i - 1].text, err.iloc[i - 1].expected, err.iloc[i -
```

```
1].predicted, "Analyse after reading the clause (ambiguous wording / similar
class)"] if err is not None and len(err) >= i else ["PENDING"] * 4
    for j, v in enumerate(row): cell(T[11].rows[i].cells[j], v)

# ---- Chapter 5 ----
settext(P[109], "The Gradio application (app.py) loads the saved checkpoint
models/best.pt once at start-up; it never retrains. Tab 1: the user types a
clause and presses Predict to see the predicted type, the model confidence "
 "(softmax probability, not legal certainty) and the top three classes. Tab 2:
the user uploads a PDF or TXT contract; the text is extracted, split into
clauses (numbering, headings, paragraphs), each clause is classified, "
 "and a table plus a downloadable CSV is shown. Clear error messages are given
for empty input, unsupported files and extraction failures. A legal disclaimer
is displayed.")
image_or_pending(T[12].rows[0].cells[0], "report_assets/app_screenshot.png",
"after running app.py, save a real screenshot as
report_assets/app_screenshot.png")
SAMPLES = ["This Agreement shall be governed by the laws of the State of
Delaware.", "Any notice under this Agreement shall be in writing and sent by
registered mail to the address of the party.",
           "This Agreement may be executed in two or more counterparts, each of
which is an original.", "If any provision is held unenforceable, the remaining
provisions remain in effect.",
           "No waiver of any breach shall be deemed a waiver of any later
breach."]
ap = json.load(open("report_assets/app_samples.json")) if
os.path.exists("report_assets/app_samples.json") else {}
for i, s in enumerate(SAMPLES, 1):
    cell(T[13].rows[i].cells[1], s); cell(T[13].rows[i].cells[2], ap.get(s,
"PENDING")); cell(T[13].rows[i].cells[3], "PENDING")

# ---- Chapter 6 ----
settext(P[117], (f"A GPT-2 (124M) based 10-class clause classifier with a new
classification head was built, trained and evaluated. On the held-out test set
it achieved accuracy {N(a_, 'accuracy')} and macro-F1 {f1} "
  f"(before fine-tuning: {N(b_, 'accuracy')} and {N(b_, 'macro_f1')}). A Gradio
application classifies single clauses and whole contracts. Objectives 1–3 are
met as shown in Table 4.2.") if a_ else
  "The complete pipeline (data preparation, GPT-2 classifier, training,
evaluation, application) is implemented. Final numbers are PENDING until
train.py is run; update this paragraph with the real accuracy and macro-F1.")
settext(P[120], "Only the 10 most frequent clause types are supported; other
clause types will be forced into one of these 10 classes.")
settext(P[121], "Clauses are cut to 256 tokens, the model is small (124M),
LEDGAR is US-English contract text, and similar classes (e.g. waiver vs.
amendment) can be confused. Predictions may be wrong and are not legal advice.")
n = copy.deepcopy(P[121]._element); P[121]._element.addnext(n); from
docx.text.paragraph import Paragraph
settext(Paragraph(n, P[121]._parent), "AI-tool declaration: an AI coding
assistant (Claude) was used to generate the project code and draft this report;
the student is responsible for understanding and explaining all code.")
settext(P[124], "Use GPT-2 medium or unfreeze more layers, and train on more
LEDGAR classes with more data."); settext(P[125], "Add OCR for scanned PDFs, a
better clause splitter and show the most similar training clauses as an
explanation.")

# ---- references ----
refs = ['[4]  M. Tuggener, P. von Däniken, T. Peetz and M. Cieliebak, “LEDGAR: A
Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in
Contracts,” in Proc. 12th Language Resources and Evaluation Conference (LREC),
2020, pp. 1235–1241.',
        '[5]  I. Chalkidis et al., “LexGLUE: A Benchmark Dataset for Legal
Language Understanding in English,” in Proc. 60th Annual Meeting of the ACL,
2022. Available: https://huggingface.co/datasets/coastalcph/lex_glue',
        '[6]  A. Paszke et al., “PyTorch: An Imperative Style, High-Performance
Deep Learning Library,” in Advances in Neural Information Processing Systems
(NeurIPS), 2019.',
        '[7]  A. Abid et al., “Gradio: Hassle-Free Sharing and Testing of ML
```

```
Models in the Wild," arXiv:1906.02569, 2019.']
settext(P[131], refs[0]); settext(P[132], refs[1])
last = P[132]
for t in refs[2:]:
    n = copy.deepcopy(last._element); last._element.addnext(n); last =
Paragraph(n, last._parent); settext(last, t)

# ---- appendix ----
settext(P[138], "Main file: train.py (project folder MP-C38); notebook: MP-
C38_clause_classifier.ipynb")
settext(P[139], "Python 3.10+, torch>=2.1, tiktoken, datasets==2.21.0, scikit-
learn, gradio (full list: requirements.txt; exact versions are printed by pip in
Colab)")
for i, t in zip(range(142, 146), ["pip install -r requirements.txt", "python
data_preparation.py   (downloads LEDGAR and creates data/*.csv)",
                                  "python train.py   (trains, saves
models/best.pt, results and charts in report_assets/); evaluate again with:
python evaluate.py",
                                  "python app.py   (opens the Gradio
application; tests: pytest tests)"]): settext(P[i], t)
tmpl_head, tmpl_code = P[148]._element, P[149]._element
anchor = tmpl_head
files = ["utils.py", "gpt_model.py", "data_preparation.py", "train.py",
"evaluate.py", "inference.py", "app.py", "fill_report.py",
"tests/test_basic.py"]
for k, fn in enumerate(files, 1):
    h = copy.deepcopy(tmpl_head); c = copy.deepcopy(tmpl_code)
    anchor.addnext(h); h.addnext(c); anchor = c
    hp, cp = Paragraph(h, P[148]._parent), Paragraph(c, P[148]._parent)
    settext(hp, f"A.3.{k}  {fn}")
    code = open(fn, encoding="utf-8").read().rstrip("\n").split("\n")
    for r in cp.runs[1:]: r._element.getparent().remove(r._element)
    r0 = cp.runs[0]; r0.text = code[0]; r0.font.color.rgb = BLACK
    for line in code[1:]:
        r0.add_break(); r0.add_text(line)
for el in (tmpl_head, tmpl_code): el.getparent().remove(el)
for i in range(150, 153): P[i]._element.getparent().remove(P[i]._element)

# ---- remove grey guidance paragraphs ----
def grey(p): return p.runs and all((r.font.color is not None and
r.font.color.type is not None and str(r.font.color.rgb) == "7F7F7F" and
r.font.italic) for r in p.runs)
for p in d.paragraphs:
    if grey(p) and not p.text.startswith(("Complete code repository", "Notebook
link")) and "Courier" not in str(p.runs[0].font.name):
p._element.getparent().remove(p._element)
d.save(dst)
left = [p.text[:70] for p in Document(dst).paragraphs if re.search(r"\[[^\]]
{2,}\]", p.text) and not p.text.startswith("[")][:0]
print("saved", dst)
```

A.3.9 tests/test_basic.py

```
import os, sys, pandas as pd, pytest
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
from utils import split_into_clauses, results_to_csv, extract_text
from data_preparation import build_balanced, split_data

SAMPLE = """1. Governing Law
This Agreement shall be governed by the laws of India and the courts of
Bengaluru shall have jurisdiction.

2. Notices
All notices under this Agreement shall be in writing and delivered by hand or
registered post to the parties.

(a) Counterparts. This Agreement may be signed in counterparts, each of which is
an original document."""
```

```
def test_segmentation_keeps_order():
    c = split_into_clauses(SAMPLE)
    assert len(c) == 3 and "governed" in c[0] and "notices" in c[1].lower() and
"Counterparts" in c[2]


def test_segmentation_no_numbering():
    txt = "Para one is long enough to be a clause on its own, really.\n\nPara
two is also long enough to stand alone as a clause."
    assert len(split_into_clauses(txt)) == 2


def test_extract_unsupported(tmp_path):
    f = tmp_path / "x.docx"; f.write_text("hi")
    with pytest.raises(ValueError): extract_text(str(f))


def test_csv_export():
    s = results_to_csv([dict(clause_no=1, clause_text="a, b",
predicted_type="X")])
    assert s.splitlines()[0] == "clause_no,clause_text,predicted_type"


def _fake():
    w = lambda n: "".join(chr(97 + (n // 26 ** k) % 26) for k in range(3))   #
letters only (digits are ignored by the dedupe key)
    rows = [(f"clause text {w(i)} of type {w(c * 7 + 1000)} with words {w(i * 31
+ c)}", f"L{c}") for c in range(12) for i in range(60)]
    rows += [rows[0]]                                    # a duplicate
    return pd.DataFrame(rows, columns=["text", "label_name"])


def test_balanced_and_split_integrity():
    bal, info = build_balanced(_fake(), per_class=50, seed=1)
    assert bal.label.nunique() == 10 and (bal.label.value_counts() == 50).all()
    tr, va, te = split_data(bal, 1)
    assert not (set(tr.text) & set(te.text)) and not (set(tr.text) &
set(va.text)) and not (set(va.text) & set(te.text))
    assert len(tr) == 350 and len(va) == len(te) == 75


def test_model_shapes_and_batch_prediction():
    torch = pytest.importorskip("torch"); pytest.importorskip("tiktoken")
    from gpt_model import GPTModel
    from utils import ClauseDataset, pad_batch, encode, PAD_ID
    cfg = dict(vocab_size=50257, context_length=64, emb_dim=32, n_heads=2,
n_layers=2, drop_rate=0.0, qkv_bias=True)
    m = GPTModel(cfg, out_dim=10).eval()
    ds = ClauseDataset(["short clause", "a much longer clause " * 40], [0, 1],
max_length=16)
    ids, lengths, y = ds[1]
    assert ids.shape[0] == 16 and lengths.item() == 16            # truncation
    assert ds[0][0][-1].item() == PAD_ID                           # padding
    b, l = pad_batch([encode("one two", 16), encode("three", 16), encode("four
five six", 16)])
    assert m.classify_logits(b, l).shape == (3, 10)                # one score
vector (10) per clause
```